

Name: Enrolment No:		 MANIPAL UNIVERSITY JAIPUR	
Odd Semester End Term Examination, November 2025 Faculty of Science, Technology & Architecture, School of Computer Science and Engineering Department of Information Technology B.Tech. – Information Technology SET-A (Solution)			
Course Code: INT2121		Course: Python Programming	Semester: III
Time: 03 hrs.		Max. Marks: 80	
Instructions: All questions are compulsory. Missing data, if any, may be assumed suitably. Calculator is allowed.			
SECTION A (Memory Based Questions)			
S. No.		Marks	CO
Q 1	Write the output of the following expressions. print (2 * 2 ** 2 * 2) 16 Print (2 not in 123) Error “int is not iterable”	[1+1]	INT2121.1
Q 2	What will be the output of the code blocks below? L1 = [x*x if x % 2 != 0 else x for x in range (1,10)] print(L1) L1 = [1,2,9,4,25,6,49,8,81]	[2]	INT2121.2
Q 3	State True/ False for the following statement (a) s = "hello"; s[0] = "H" will execute without error. False (b) (10,) is a valid tuple with one element. True	[1+1]	INT2121.3
Q 4	Write the output of following code try: x = int ("10") y = int ("five") except ValueError as e: print(type(e)) <class 'ValueError'>	[2]	INT2121.3
Q 5	Which method does NOT raise an error in set if element is not found? a) remove() b) discard() c) both d) neither	[2]	INT2121.2
SECTION B (Concept Based Questions)			
Q 6	a) Why are file opening modes required in file handling (2 marks)? Explain different modes with suitable examples. (.5 mark for each mode with example, explain atleast 4 modes) Solution: File-opening modes specify how a file should be accessed—whether we want to read, write, append, or create a file. They help the operating system understand: • What operation is allowed on the file (read/write/append). • Whether an existing file should be overwritten. • Whether a new file should be created if it does not exist. • Whether the file should be opened in text or binary form. Without modes, Python would not know the purpose for which the file is being accessed. <i>Different File Modes (with examples)</i> 1. r – Read mode • Opens file only for reading. • File must already exist. Example: f = open("data.txt", "r") print(f.read()) 2. w – Write mode	[2+4*.5]	INT2121.3

	• Opens the file for writing only. • Overwrites the file if it exists; creates new file if it does not. Example: f = open("data.txt", "w") f.write("Hello World") 3. a – Append mode • Opens file for adding new content at the end. • Existing data is not overwritten. Example: f = open("data.txt", "a") f.write("\nNew line added") 4. b – Binary mode • Used to read/write binary files like images, videos. Example: f = open("photo.jpg", "rb") data = f.read() 5. x – Exclusive creation mode • Creates a new file. • Raises an error if the file already exists. f = open("newfile.txt", "x") 6. t – Text mode (default) • Used for reading/writing text files. Example: f = open("notes.txt", "rt") b) Write a program in Python to read a text file (“<i>input.txt</i>”) and copy every alternate word of the file in another file (“<i>output.txt</i>”). # Read from input.txt (1 marks for reading file and 1 mark for correct file name) with open("input.txt", "r") as f1: data = f1.read().split() # Split text into words # Select every alternate word (0,2,4,...) (1 mark for selecting alternate word) alternate_words = data[::2] # Write to output.txt (1 mark for writing file correctly) with open("output.txt", "w") as f2: f2.write(" ".join(alternate_words))		
Q 7	a) Display the following pattern: (5 marks if output is correct) <pre> 1 3 2 1 9 7 5 3 1 13 11 9 7 5 3 1 </pre> # Starting numbers of each row start_values = [1, 3, 9, 13] # Number of rows rows = len(start_values) for i in range(rows): # Print leading spaces for alignment print(" " * (rows - i - 1) * 6, end="") # Print descending numbers for each row num = start_values[i] for j in range(i*2 + 1): print(num, end=" ") num -= 2 # decrease by 2 to keep odd descending values print() # move to next line	[5+3]	INT2121.1

b) How **break** is different from **continue** in Python Programming (2*1.5=3 marks)? What is the role of '**in**' operator in **for** loop? (1 mark)

Difference between break and continue in Python

1. break Statement

- The break statement **terminates the loop immediately**.
- Control comes **out of the loop** and moves to the next statement after the loop.

```
for i in range(1, 6):
```

```
    if i == 3:
```

```
        break
```

```
    print(i)
```

Output: 1 2 (Loop stops when i becomes 3)

2. continue Statement

- The continue statement skips the current iteration and moves to the next iteration of the loop.
- It does not terminate the loop.

Example:

```
for i in range(1, 6):
```

```
    if i == 3:
```

```
        continue
```

```
    print(i)
```

Output: 1 2 4 5 (Only iteration with i = 3 is skipped)

Role of the in operator in a for loop

- The in operator is used to iterate over elements of a sequence like list, string, tuple, or range.
- It tells the loop from where the values should come.

Example:

```
for x in [10, 20, 30]:
```

```
    print(x)
```

Here, *in* means: "Take each element from the list one-by-one and assign it to x."

Q 8	Consider a class named '<i>Candidate</i>' with below given member variables (properties) and method (behavior): (1 mark for declare class) (candidate_id: integer, candidate_name: String, question_marks: List of integers (for 5 questions), result: Boolean) (1 mark for correct variable name) Define a class method as '<i>findSelect()</i>' which find the result for the candidate Interviews based on correct answer marks (3 for correct answer) and return boolean <i>result</i> value as mentioned below: (2 mark for function definition and correct conditions) (i) result is <i>false</i> (NOT SELECTED) if marks obtained are below 50%. (ii) result is <i>true</i> (SELECTED), otherwise. Write a python code to create the class named '<i>Candidate</i>' as per the above description. a) Create two <i>Candidate</i> type instances (objects) of your choice and assign values to all the properties using <i>_init()</i> method. (2 marks) b) Demonstrate how you can find result of each candidate (selected or not selected) based on instances created in Part (a) above. (2 marks) Solution: <pre>class Candidate: def __init__(self, candidate_id, candidate_name, question_marks): self.candidate_id = candidate_id self.candidate_name = candidate_name self.question_marks = question_marks # list of integers for 5 questions self.result = False # default def findSelect(self): total_marks = sum(self.question_marks) # sum of all marks max_marks = 5 * 3 # 5 questions × 3 marks each = 15 percentage = (total_marks / max_marks) * 100 if percentage >= 50: self.result = True else: self.result = False return self.result # Creating two candidate objects c1 = Candidate(101, "Rahul Sharma", [3, 3, 0, 3, 3]) # total = 12 c2 = Candidate(102, "Neha Patel", [0, 3, 0, 0, 3]) # total = 6 # Check results print(c1.candidate_name, "Selected:", c1.findSelect()) print(c2.candidate_name, "Selected:", c2.findSelect())</pre>	[4 + 2+ 2]	INT2121.4
Q 9	a) Write short note on the following exception handling blocks with suitable example: (i) try (1 mark) (ii) except (1 mark) (iii) else (1 mark) (iv) finally (1 mark) Solution: (i) try Block - The try block contains the code that may raise an exception. - Python tests this block first; if an error occurs, control goes to except. <pre>try: x = int("10") # valid y = int("abc") # may raise error</pre>	[4 + 2]	INT2121.3

	(ii) except Block - The except block is executed only when an exception occurs in the try block. - It is used to handle the error and prevent program crashes. <pre>try: num = int("abc") except ValueError: print("Conversion failed! Invalid number.")</pre> iii) else Block - The else block runs only if no exception occurs in the try block. - It is used for code that must execute when everything goes correctly. <pre>try: num = int("20") except ValueError: print("Error!") else: print("Conversion successful:", num)</pre> (iv) finally Block - The finally block always executes, whether an exception occurs or not. - It is used for cleanup actions, such as closing files or releasing resources. <pre>try: f = open("data.txt", "r") data = f.read() except FileNotFoundError: print("File not found!") finally: print("Execution completed (file closed if opened).")</pre> b) Write a python program that executes division and handles an <i>ArithmeticError</i> exception if there is an arithmetic error. (1 mark for correct input and 1 mark for handling exception correctly) <pre>try: a = int(input("Enter numerator: ")) b = int(input("Enter denominator: ")) result = a / b print("Result =", result) except ArithmeticError: print("Arithmetic Error occurred! (Possible division by zero)")</pre>		
SECTION-C (Analytical Based Questions)			
Q 10	Given a list of tuples where each tuple represents a student's name and their marks in three subjects, write a Python program that: (2 mark for list declaration) - Prints each student's average marks (3 marks) - Displays the student with the highest average (3 marks) - Print names of students who scored more than 80 in all three subjects (2 marks) Solution: <pre>students = [("Rahul", 85, 78, 92), ("Neha", 90, 88, 91), ("Amit", 75, 80, 79), ("Priya", 82, 85, 89), ("Karan", 95, 96, 94)] highest_avg = 0 top_student = "" print("Average marks of each student:") for s in students: name, m1, m2, m3 = s</pre>	[2+3+ 3+2]	INT2121.2

	<pre> avg = (m1 + m2 + m3) / 3 print(f'{name}: {avg:.2f}') # find highest average if avg > highest_avg: highest_avg = avg top_student = name print ("\nStudent with highest average:", top_student) print ("\nStudents scoring more than 80 in all subjects:") for s in students: name, m1, m2, m3 = s if m1 > 80 and m2 > 80 and m3 > 80: print(name) </pre>		
Q 11	Consider a 2-D NumPy array representing the daily temperature readings recorded in different cities. Each row corresponds to a specific day, and each column represents the temperature recorded in a particular city. [2 marks] Solution: a) Write a Python program to calculate the average temperature of each city across all days. [2 marks] b) Determine the day with the highest overall temperature (sum of temperatures recorded in all cities). [2 marks] c) Identify the city with the lowest average temperature over the days. [2 marks] d) Compute the percentage contribution of each day's temperature to the total temperature recorded in the entire dataset. [2 marks] Input Trading Sample: <pre> [[28, 31, 30], [33, 32, 34], [30, 26, 28]] </pre> <pre> import numpy as np # Input 2-D NumPy array temps = np.array([[28, 31, 30], [33, 32, 34], [30, 26, 28]]) # a) Average temperature of each city (column-wise mean) city_avg = np.mean(temps, axis=0) print ("Average temperature of each city:", city_avg) # b) Day with highest overall temperature (sum across columns) day_sum = np.sum(temps, axis=1) highest_day = np.argmax(day_sum) + 1 # +1 to show Day number print ("Day with highest overall temperature: Day", highest_day) # c) City with lowest average temperature lowest_city = np.argmin(city_avg) + 1 # +1 for city number print ("City with lowest average temperature: City", lowest_city) # d) Percentage contribution of each day's temperature total_temp = np.sum(temps) percentage = (day_sum / total_temp) * 100 print("Percentage contribution of each day:", percentage) </pre>	[2+2+ 2+2+2]	INT2121.5

SECTION-D (Case Study/Application based Question)

Q 12

Consider the below given **Book** schema/structure of a dictionary:

Book {B-Name: string, Author: string, Price: integer, Pages: integer, Copies-sold: integer, Revenue: float}

a) Write a python code to create **Book** dictionary to maintain given book' information (**take user input**). (4 marks)

B-Name	Author	Price (₹)	Pages	Copies-sold	Revenue (₹)
Python Made Easy	John Smith	450	320	27	0.0
Learn C++	David Miller	500	420	5	0.0
Data Structures	Raj Mehta	625	545	12	0.0
AI Revolution	Priya Sharma	810	670	19	0.0

a) Create Book dictionary using user input for n books (

```
books = {}
```

```
n = int(input("Enter number of books: "))
```

```
for i in range(n):
```

```
    print(f'\nEnter details of Book {i+1}:')
```

```
    name = input("Book Name: ")
```

```
    author = input("Author: ")
```

```
    price = int(input("Price: "))
```

```
    pages = int(input("Pages: "))
```

```
    copies = int(input("Copies Sold: "))
```

```
# Store inside main dictionary
```

```
books[name] = {
```

```
    "Author": author,
```

```
    "Price": price,
```

```
    "Pages": pages,
```

```
    "Copies-sold": copies,
```

```
    "Revenue": 0 # will update later
```

```
}
```

b) # b) Define a function to calculate the *Revenue* of each book and update the dictionary.

Revenue = Price × Copies-Sold

Function to calculate revenue and update dictionary (3 marks)

```
def calculate_revenue(book_dict):
```

```
    for book_name, details in book_dict.items():
```

```
        revenue = details["Price"] * details["Copies-sold"]
```

```
        details["Revenue"] = revenue # update revenue
```

```
# Call function
```

```
calculate_revenue(books)
```

c) Write code to find and display the *Book Name* and *Author* who earned the maximum *Revenue*.

Find book with maximum revenue (3 marks)

```
max_book = max(books, key=lambda x: books[x]["Revenue"])
```

```
print("\nBook with Maximum Revenue:")
```

```
print("Book Name:", max_book)
```

```
print("Author:", books[max_book]["Author"])
```

```
print("Revenue:", books[max_book]["Revenue"])
```

[4+ 3 +3]

INT2121.2

Q 13	Write python code to do following tasks: 1. Convert the dictionary Book created in Q12 into DataFrame Book_Dict. <code>Book_Dict = pd.DataFrame.from_dict(books, orient='index')</code> 2. Find the <i>book names</i> whose <i>Revenue</i> is more than ₹1,00,000 and store the result in a Series/DataFrame. <code>High_Revenue = Book_Dict[Book_Dict["Revenue"] > 100000]</code> 3. Delete the record from <i>Book_Dict</i> if the <i>Price</i> of the book is below ₹500.50 crores. <code>Book_Dict = Book_Dict[Book_Dict["Price"] >= 500.50]</code> 4. Delete the field/column "<i>Pages</i>" from <i>Book_Dict</i>. <code>Book_Dict = Book_Dict.drop(columns=["Pages"])</code> 5. Add a new record into <i>Book_Dict</i> as ('ML Mastery', 'A. Verma', 950, 720, 350, 0.0) <code>Book_Dict.loc["ML Mastery"] = { "Author": "A. Verma", "Price": 950, "Pages": 720, "Copies-sold": 350, "Revenue": 0.0 }</code>	[2+2+2+2+2]	INT2121.5
--------------------	---	--------------------	------------------